The vibes that I summoned…

Thoughts on working with AI agents.

Chris Pahl · 2026

The Spectrum (on Reddit)

Full manual
every keystroke yours

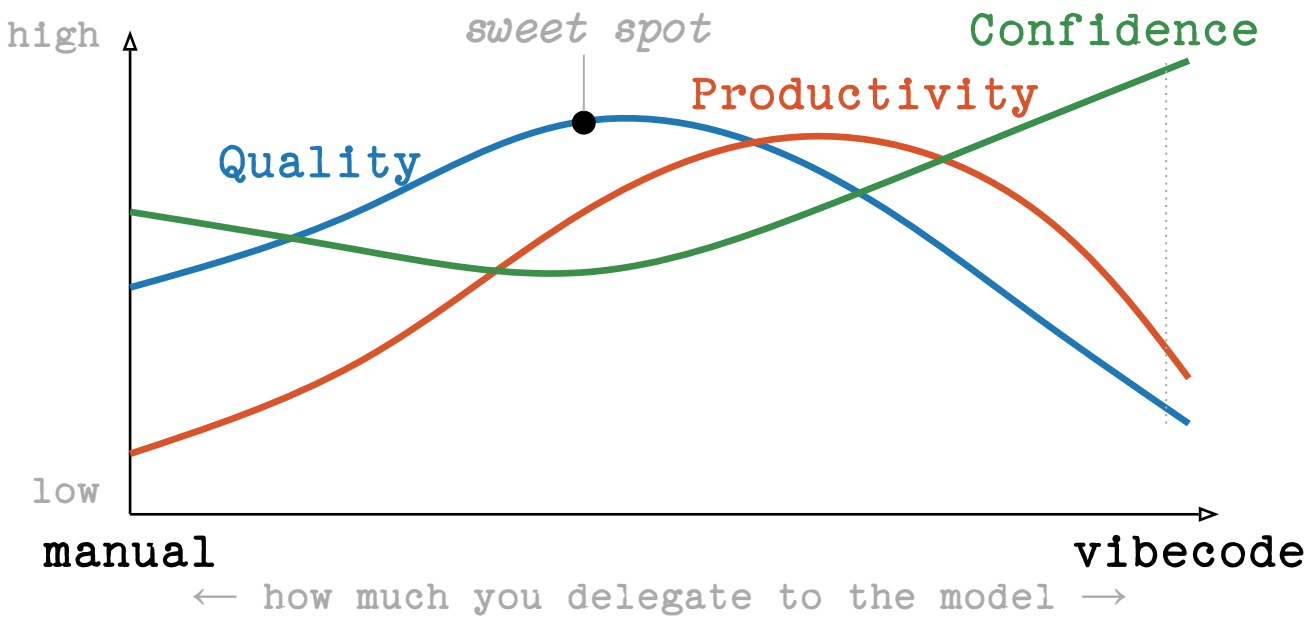
Full vibecode
just make it work

⟵ more control · more understanding | more productivity · more delegation ⟶

Using GenAI is a tradeoff between control and productivity.

It’s not black & white.

My take: Quality vs Productivity



Quality peaks in the middle. Productivity follows it down. Confidence diverges from both -> Dunning-Kruger zone!

13%

But there's already data:

-19%

slower with AI

experienced devs · own mature repos · predicted +24%

METR · RCT · 2025

~40%

insecure programs

Copilot output across 89 security-relevant scenarios

NYU CCS · 2021

8×

more duplicated code

block-level clones up · refactoring 24% → 10%

GitClear · 211M LoC · 2024

-7.2%

delivery stability

drop with AI adoption · 39% distrust AI code

DORA / Google · 2024

29.5%

Python with CWEs

AI-generated snippets containing known vulnerabilities

Fu et al. · ACM TOSEM · 2025

↓

trust = less scrutiny

more trust in GenAI ⟹ less critical thinking applied

Lee et al. · Microsoft + CMU · CHI 2025

Good use cases

rubber-ducking ideation

explain unfamiliar code test generation

pair programming boilerplate refactoring assist

learning accelerator documentation drafts naming things

summarisation regex / SQL crafting edge-case brainstorming

translation code review companion mock data / fixtures

error decoding onboarding ramp-up log analysis proofreading

automation

Risks for all

ecological cost

Convincing hallucinations

schools

licensing

Atrophy

concentration

misinformation at scale

Prompt injection & MCP

content degradation

No juniors hired

hardware crisis

operator bias

recursive collapse

economic transfer

data privacy

cyberattack automation

communities thinning out

Risks for us

ecological cost

Convincing hallucinations

schools

licensing

Atrophy

concentration

misinformation at scale

Prompt injection & MCP

content degradation

No juniors hired

hardware crisis

operator bias

recursive collapse

economic transfer

data privacy

cyberattack automation

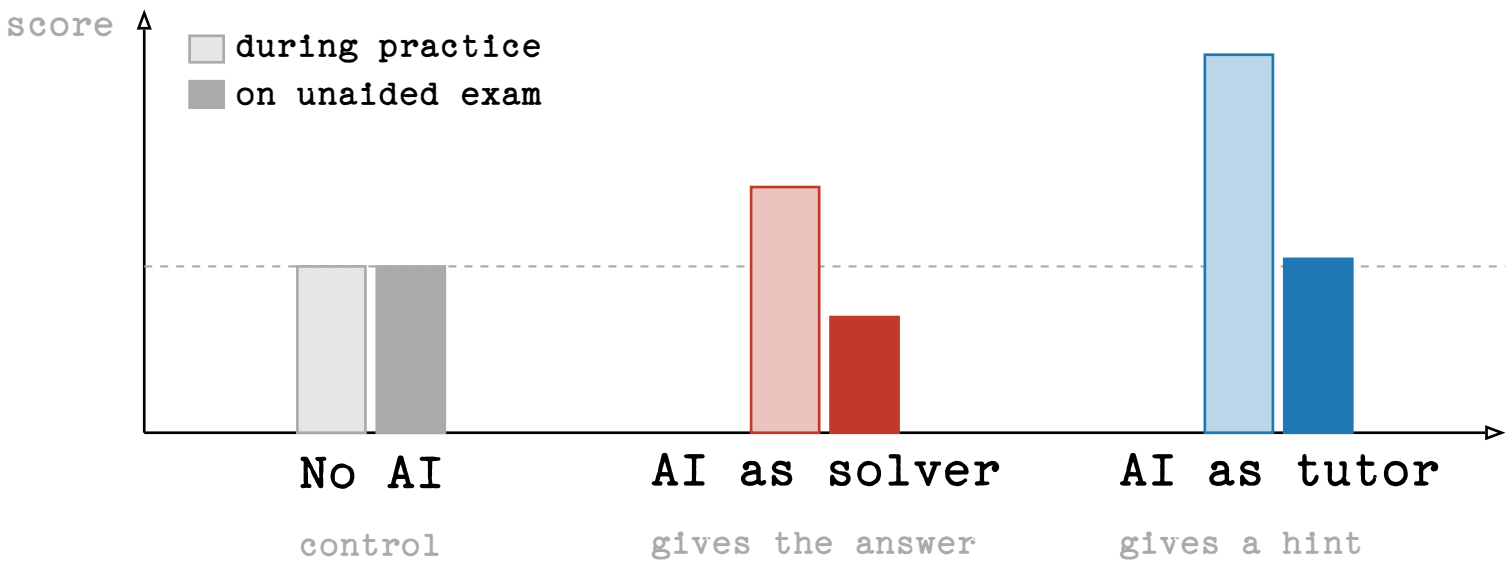
communities thinning out

Exhibit A: schools.

Maybe it's not so much about the tool —
but about how we use it?

33%

Same tool, opposite outcomes



Bastani et al., *Generative AI Can Harm Learning*, SSRN 2024

Are *you* team tutor or team solver?

Keeping up with Claude

Our team's Claude Code account · April 2026

98.7%

suggestions accepted as-is

1.3% rejected. Is that a review process?

27,757

lines accepted last month

≈ 925 LoC/day · careful review ≈ 100–200 LoC/hour

Reviewing 27,757 lines carefully ≈ 138 h. A working month ≈ 176 h.

Either we're 7× faster than the industry — or we didn't review.

Human cognitive biases

Automation bias

We accept machine suggestions more readily than human ones.

click accept · review later · maybe

Anchoring

The first suggestion shapes the solution space — even when it's wrong.

the model frames the problem before you do

Confirmation bias

We hear what we already believe.

the prompt contains the answer we want

Authority bias

Eloquent + fast + confident reads as expert.

fluency ≠ correctness

Dunning–Kruger

We mistake competence we observe for competence we have.

"this is what I would have written"

Illusion of explanatory depth

We think we understand — until asked to explain.

"yes, I read the diff" → bug

Statistical model biases

Sycophancy

It's easy to talk the model out of a correct answer.

echo chambers — but for code

Historical / representation bias

Trained on the web — heavily modern, English, Western.

defaults track what's common, not what's right

Attention bias

First and last things dominate; the middle evaporates.

rules buried in long context get ignored

Omission bias

Rare and novel answers get suppressed by common ones.

the model is bad at being weird

Hen & Egg

- If you don’t know what good software looks like -
  how do you write the right prompt?
- If you can’t understand what the model just generated -
  how do you verify it?
- If you can’t code (anymore) -
  how can you understand the diffs?

Humans are still very much required.

How do *you* keep up with Claude?

Five habits that helped me

1. Think first, generate then.
2. Split the work - keep some of it manual.
3. Have a real verification strategy.
4. You only get replaced if you make yourself replaceable.
5. Treat the AI as a colleague.

1. Think first, generate then

Don’t let the model set the anchor.

- Sketch the SQL query yourself,
- Then ask the model to review and optimise.
- Write the function signature and the docstring.
- Then let the model fill the body.
- You set the anchor; the model refines.

2. Split the work

The parts you write are the parts you understand.

- Leave the manual labor work to the model (e.g. build systems)
- Keep doing at least some smaller tasks yourself.
- I suggest: Do most of critical production code yourself.
- Tests, Build system, scripts, boilerplate: Default to Claude.

3. Have a real verification strategy

- Before you accept a generated change, name the signal that would tell you it’s wrong.
- Find other ways if the diff is too big:
  - Property-based tests, fuzzing (e.g. *noise* for Dart).
  - Type checkers, linters, static analysers.
  - Replay against real production data.
- If you can’t say how you’d notice the bug, you don’t have a verification strategy - you have hope.

Example: Noise library for Dart. I don’t speak Dart.

4. You only get replaced if you make yourself replaceable.

You only get replaced if you make yourself replaceable.

- The replaceable parts of your job are the parts where you're already a slow autocomplete.
- Spend the time AI saves you on the parts no model can do: judgement, context, design.
- Code was the source of truth before, that's changing.
- Now you have more time to think about design and purpose.
- If you just operate Claude you are replaceable.

5. Treat the AI as a colleague

Not an oracle, not a junior, not magic.

- Push back. Ask for alternatives. Disagree.
- It’s a very eager, sometimes rather junior colleague with seemingly infinite power.
- If you wouldn’t accept a colleague’s PR with the same reasoning, don’t accept the model’s.

Extra: Software as Theory Building

Peter Naur, 1985

- The code is a by-product. What you're really building is the theory – your team's shared mental model of the problem.
- Documentation captures the artefact, not the theory. The bugs, the dead ends, the "why not this?" decisions live in people's heads.
- This is why handovers fail. The theory doesn't transfer cleanly.

Peter Naur, 1928–2016
Photo: Wikimedia Commons

What that means for AI

- The model has no theory. It has the artefact and the context you give it.
- “Write tests for this file” → it freezes today’s behaviour, bugs included.
- Your job is still to build the theory. The AI helps you write the code that follows from it.

Practical:

- Keep a design doc / decision log. The why, not just the what.
- CLAUDE.md is size-limited — use it for headlines, link out for depth.

Questions or Disagreements?

Homework:

Read SOFTWARE AS THEORY BUILDING.